UNIVERSIDAD AUTÓNOMA DE CAMPECHE

REPORTE DE PRÁCTICA
LOGIN SQL INJECTION

2DO PARCIAL | EXPLOTACIÓN CONTROLADA Y CORRECCIÓN

Asignatura: Seguridad Informática

Docente: Sergio A Noh Puch

Alumno: Sergio David Foster Medina

Fecha: 27 de octubre de 2026

Tecnologías: Node.js, Express, SQLite3, HTML y JavaScript

Objetivo

Identificar y demostrar una vulnerabilidad de inyección SQL en un formulario de inicio de sesión, corregirla mediante una consulta parametrizada y comprobar que la autenticación normal permanece operativa mientras se rechazan los payloads de prueba.

Descripción del entorno

La aplicación se ejecuta localmente en http://localhost:3000. El frontend envía usuario y contraseña a un servidor Express; este consulta una base SQLite de laboratorio, creada en memoria, que incluye las credenciales admin/admin y juan/1234.

Las pruebas se realizaron sobre el proyecto didáctico proporcionado para la materia. Las capturas de este documento corresponden a ejecuciones reales en el navegador del alumno.

1. Identificación de la vulnerabilidad

En la versión original, los valores recibidos se insertaban directamente en el texto SQL. Esto permitía que caracteres como la comilla simple y el comentario -- alteraran la lógica de autenticación.

```
const query = `SELECT * FROM users WHERE username = '${username}' AND password = '${password}'`;
```

El payload admin' -- cierra la cadena del usuario y convierte en comentario la comprobación posterior de contraseña. Por ello la versión vulnerable devolvía admin aunque la contraseña introducida fuera 123456.

```
SELECT * FROM users
WHERE username = 'admin' -- ' AND password = '123456'
```

2. Payloads utilizados y comportamiento

Payload en el campo usuario	Propósito	Evidencia del navegador
admin' --	Omitir la condición de contraseña.	Éxito antes; rechazado después.
' OR 1=1 --	Introducir una condición siempre verdadera.	Rechazado en la versión corregida.

En ambos casos se empleó la contraseña 123456. El primer payload fue documentado tanto antes como después de la corrección; del segundo se cuenta con captura posterior a la corrección.

3. Corrección implementada

Se sustituyó la interpolación por parámetros enlazados. La instrucción SQL mantiene su estructura y SQLite recibe el usuario y la contraseña como valores, sin interpretarlos como sintaxis ejecutable.

```
const query =
  "SELECT id, username FROM users WHERE username = ? AND password = ?";

db.get(query, [username, password], (err, row) => {
  // Manejo del error y del resultado de autenticación
});
```

Se mantuvo la ruta del formulario POST /login-vulnerable para conservar compatibilidad con index.html y permitir comparar los resultados sin modificar la interfaz. El nombre de la ruta es heredado de la práctica; la consulta corregida usa parámetros.

Alcance de la corrección: mitigación de SQLi en este flujo de login. Para un sistema de producción también sería necesario almacenar hashes de contraseñas y añadir controles de sesión, intentos y registro seguro de errores.

4. Evidencia: antes y después de la corrección

4.1 Antes: inicio de sesión no autorizado

Se ingresó admin' -- como usuario y 123456 como contraseña. Pese a que no era la contraseña de admin, la respuesta fue ok: true y user: admin.



Figura 1. Versión vulnerable: SQLi con comentario permite acceder como admin.

4.2 Después: mismo intento rechazado

Se repitió admin' -- con 123456 tras aplicar los parámetros. La respuesta cambió a ok: false y Credenciales inválidas.

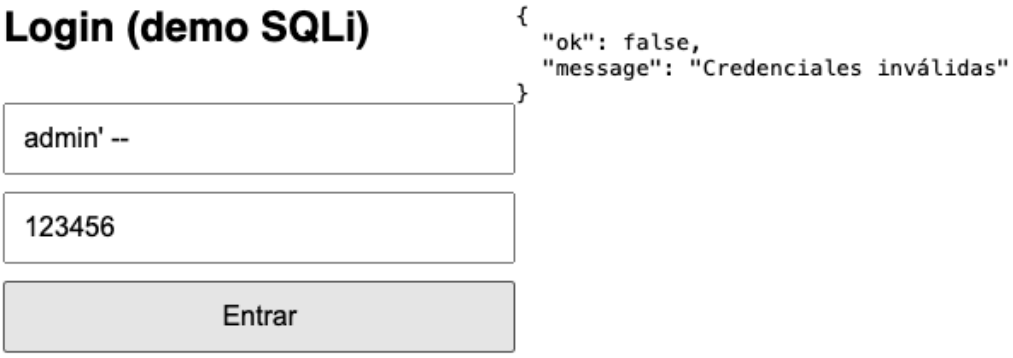


Figura 2. Versión corregida: el mismo payload ya no autentica.

5. Evidencias adicionales en el login corregido

5.1 Rechazo de tautología SQL

El usuario ' OR 1=1 -- con contraseña 123456 produjo ok: false. La condición no fue incorporada a la consulta SQL.

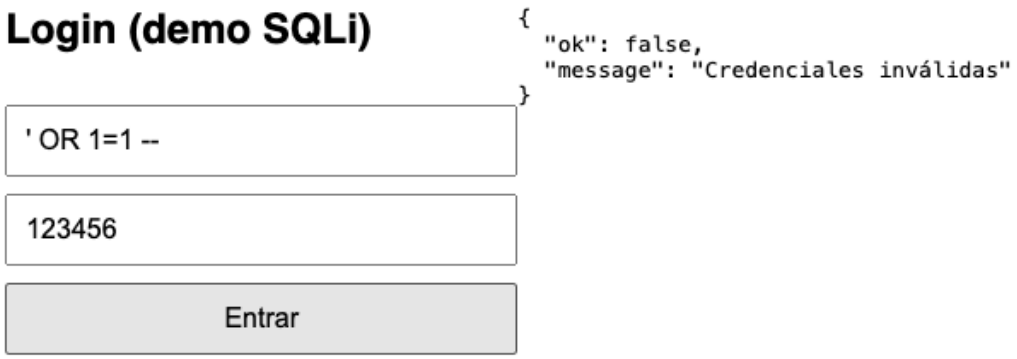


Figura 3. La inyección OR 1=1 es rechazada.

5.2 Autenticación legítima: admin

El ingreso normal de admin / admin devolvió ok: true, lo que demuestra que la corrección no bloquea el acceso autorizado de esta cuenta de laboratorio.



Figura 4. Inicio de sesión legítimo con admin / admin.

6. Evidencia de la segunda cuenta legítima

El usuario juan inició sesión utilizando su contraseña correcta 1234; el servidor devolvió ok: true y user: juan.

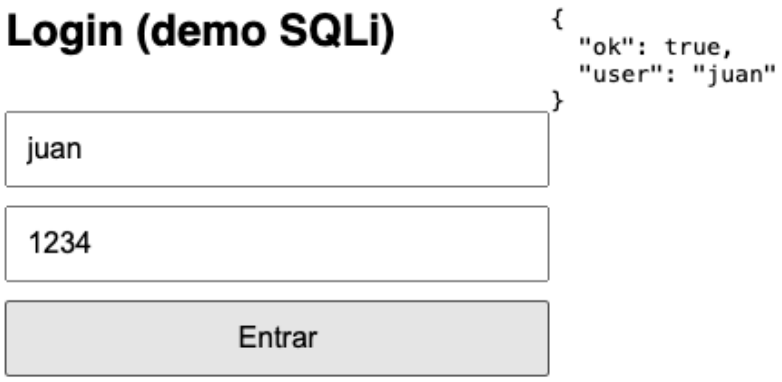


Figura 5. Inicio de sesión legítimo con juan / 1234.

7. Resumen de resultados

Prueba	Antes de corregir	Después de corregir
admin / admin	No documentado con captura	Permitido (captura).
admin' -- / 123456	Permitido (captura).	Rechazado (captura).
' OR 1=1 -- / 123456	No documentado con captura	Rechazado (captura).
juan / 1234	No documentado con captura	Permitido (captura).

La evidencia demuestra que la corrección evita las dos inyecciones probadas en el navegador y preserva la autenticación con las dos cuentas de ejemplo. No se afirma que estos ensayos constituyan una auditoría de seguridad completa.

8. Pull Request de GitHub

La consigna también solicita un Pull Request de la versión segura. Para completarlo se debe subir el cambio de server-vulnerable.js a una rama del repositorio, abrir el Pull Request hacia el repositorio correspondiente y anexar la captura o enlace del PR. A la fecha de preparación de este reporte, ese paso no se encuentra acreditado mediante evidencia aportada.

9. Conclusiones

La prueba inicial confirmó un bypass de autenticación por inyección SQL: una entrada especialmente construida eliminaba el requisito de la contraseña. Al utilizar marcadores ? y entregar los valores mediante parámetros, los intentos de modificar la consulta dejaron de producir acceso. Las pruebas legítimas con admin y juan mostraron que el formulario siguió funcionando. La práctica ilustra que separar datos y sentencias SQL es una medida fundamental para proteger la autenticación.

Nota sobre las imágenes: se muestran recortes del área relevante de capturas originales aportadas por el alumno. Solo se retiraron márgenes de la ventana para mejorar la legibilidad; se conservaron los campos introducidos y la respuesta que devolvió el servidor.